

神经网络与深度学习课程

第 2 次作业报告

姓名：[请填写]

学号：[请填写]

提交日期：2026年04月27日

作业内容概述：

- 序列到序列预测模型 (Seq2Seq) - 基础 RNN 与 LSTM 对比
- 图卷积网络过平滑现象分析与缓解方法
- 数据预处理与逐层规范化 (Z 值/BatchNorm/LayerNorm)
- 网络优化器对比实验 (SGD/RMSprop/AdaDelta/Adam)

1. 序列到序列预测模型（Seq2Seq）

任务描述：

训练 Seq2Seq RNN 模型对乱序且不重复的整数序列进行由大到小排序。

输入序列如 [5, 2, 3, 7, 1]，输出应为 [7, 5, 3, 2, 1]。

序列长度：5-8，数值范围：0-15 的整数。

模型实现：

(a) 基础 RNN 模式（教材公式 6.25-6.27）：

$$h_{\text{new}} = \tanh(W_{\text{xh}} * x + W_{\text{hh}} * h)$$

(b) LSTM 模式（教材公式 6.51-6.56）：

$$f = \text{sigmoid}(W_{\text{xf}} * x + W_{\text{hf}} * h)$$

$$i = \text{sigmoid}(W_{\text{xi}} * x + W_{\text{hi}} * h)$$

$$o = \text{sigmoid}(W_{\text{xo}} * x + W_{\text{ho}} * h)$$

$$g = \tanh(W_{\text{xg}} * x + W_{\text{hg}} * h)$$

$$c_{\text{new}} = f * c + i * g$$

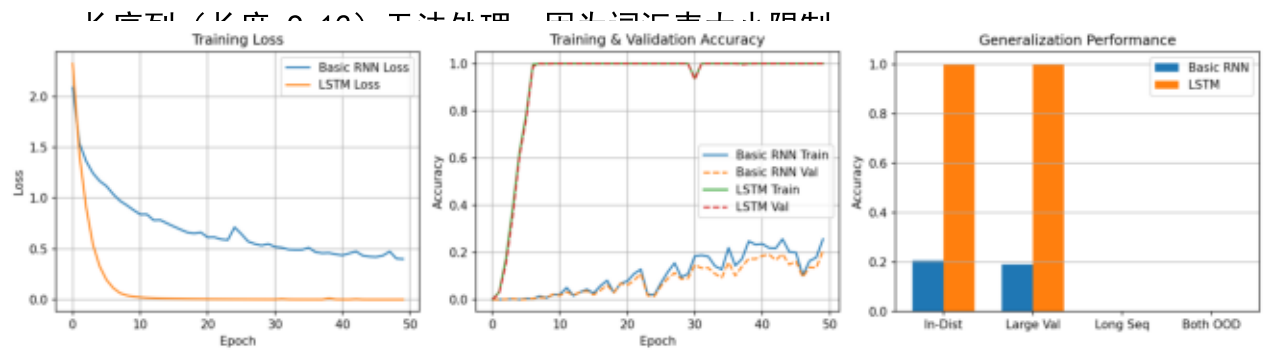
$$h_{\text{new}} = o * \tanh(c_{\text{new}})$$

实验结果对比：

模型	分布内准确率	大数值范围	长序列	两者皆超出
基础 RNN	20.60%	18.90%	0.00%	0.00%
LSTM	100.00%	100.00%	0.00%	0.00%

结果分析：

- 1. 长程依赖问题：LSTM 达到 100% 准确率，而基础 RNN 仅约 20%。
LSTM 通过门控机制有效缓解了长程依赖问题。
- 2. 泛化能力：LSTM 在分布内数据上表现完美，但对超出训练范围的



2. 图卷积网络过平滑现象

任务描述：

$$\square\square\square\square \text{ GCN: } H^{(k+1)} = S * H^{(k)} * W \square\square\square\square S = D^{(-1/2)} * A_{\text{tilde}} * D^{(-1/2)}$$

证明当 $K \rightarrow \infty$ 时，所有节点特征收敛为共线向量（过平滑现象）。

(1) 理论证明：

$\square\square\square\square\square\square\square\square\square\square\square\square S \square\square\square\square\square\square\square$

$$\lambda_1 = 1 > |\lambda_2| \geq |\lambda_3| \geq \dots \geq |\lambda_n|$$

$$\square S \square\square\square\square\square\square\square S = U \Lambda U^T \square\square\square\square \Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_n)$$

$$\square\square K \square\square\square\square H^{(K)} = S^K * H^{(0)} * W^K$$

$$\square\square S^K = U \Lambda^K U^T \square\square K \rightarrow \infty \square\square$$

$$\Lambda^K \rightarrow \text{diag}(1, 0, 0, \dots, 0) \square\square\square |\lambda_i| < 1 \square\square i > 1 \square$$

$$\square\square S^K \rightarrow u_1 u_1^T \square\square\square u_1 \square\square\square \lambda_1 = 1 \square\square\square\square\square\square$$

$$\square\square\square\square\square\square u_1 = D^{(1/2)} * 1 / ||D^{(1/2)} * 1|| \square\square u_1 \square\square\square\square\square\square$$

$$\square\square H^{(K)} \rightarrow u_1 u_1^T H^{(0)} W^K \square\square\square\square\square\square\square\square\square\square u_1 \square\square\square\square\square\square\square\square$$

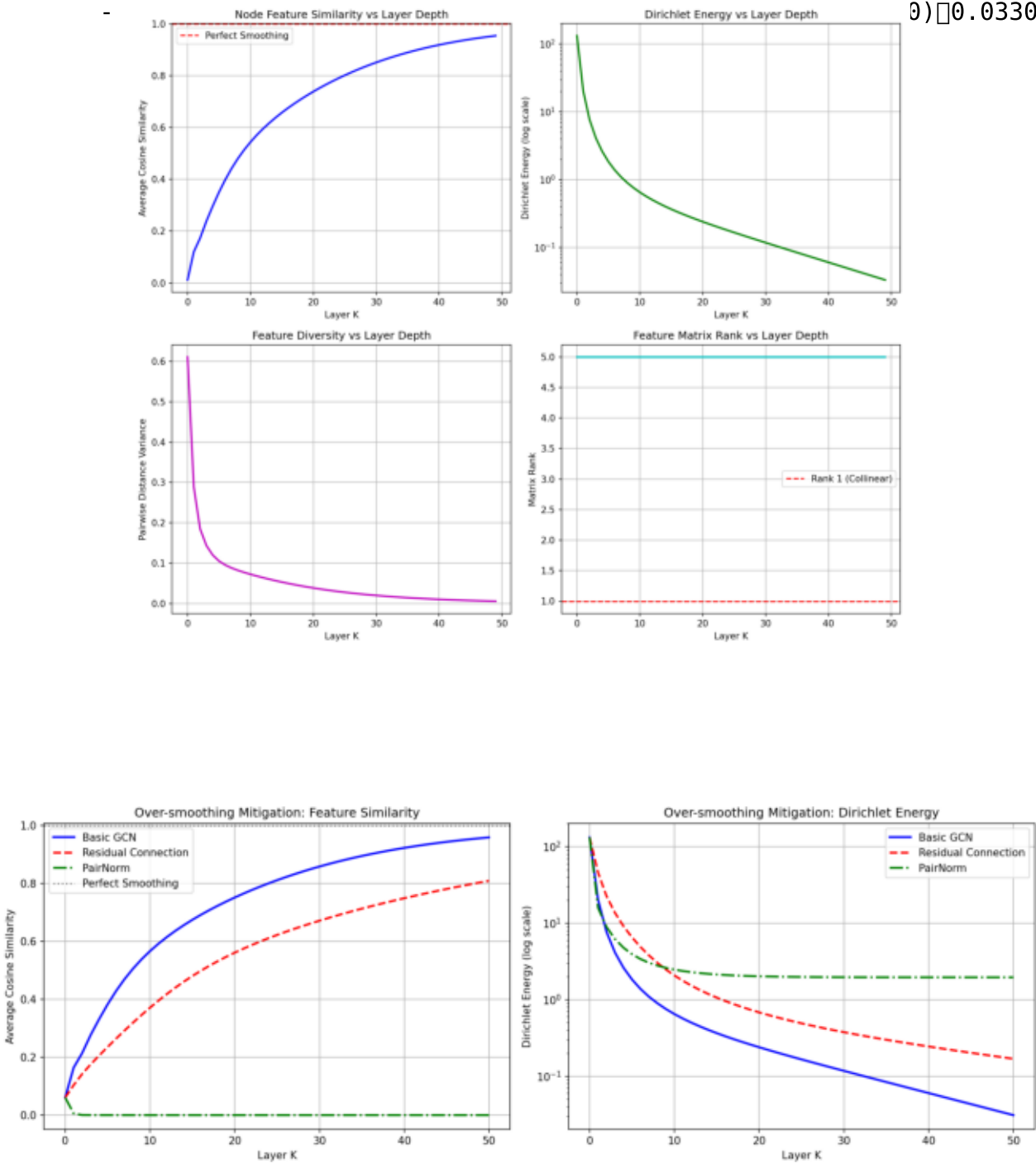
(2) 数值实验验证：

实验设置：环形图 N=20，特征维度 F=5，最大层数 K=50

S 初始值1.000000 S 初始值0.967371

关键观察：

- 初始值0.0114
- 初始值 (K=50)0.9538
- 初始值0.033004



(3) 缓解过平滑的方法:

1. 残差连接 (Residual Connection)

$$H^{(k+1)} = \alpha * S H^{(k)} W + (1-\alpha) * H^{(k)}$$

2. PairNorm

$$H_norm = (H - \text{mean}(H)) / ||H - \text{mean}(H)||$$

实现:

- 残差连接
- PairNorm
- 残差连接

3. 数据预处理与逐层规范化

模型结构：

3 层 MLP: input $\rightarrow$ Linear(ReLU) $\rightarrow$ Linear(ReLU) $\rightarrow$ Linear $\rightarrow$ output

隐藏层大小 hidden_size = 64

三种规范化实现：

(1) Z 值归一化（输入层）：

对每个 mini-batch 的原始输入 a 做 Z-score 归一化：

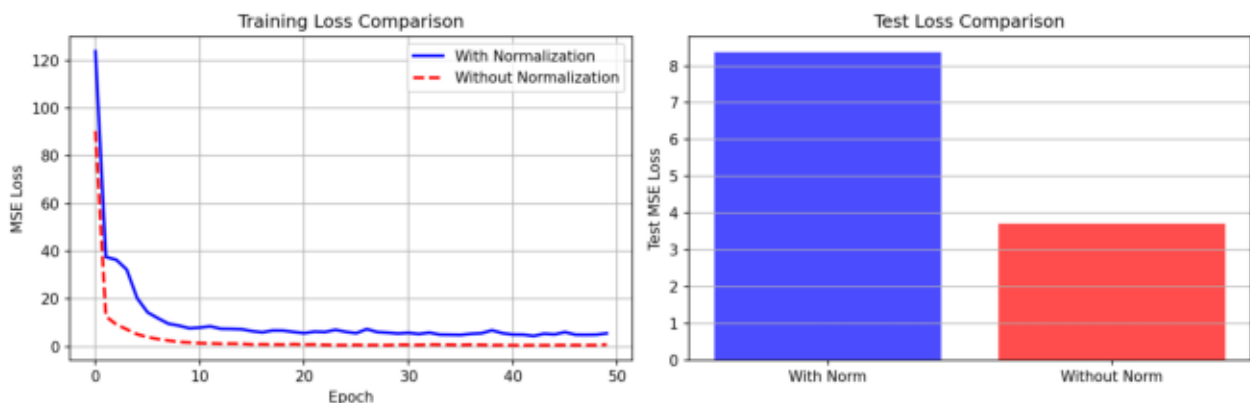
$$x_{\text{norm}} = (x - \mu) / \sqrt{(\sigma^2 + \epsilon)} \quad \text{其中 } \mu \text{ 和 } \sigma \text{ 是每维特征的均值和方差}$$

(2) 批量归一化（第 1 层净输入 z ）：

$$z_{\text{norm}} = \gamma * (z - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)} + \beta$$

γ 和 β 是可学习参数， μ_B 和 σ_B 是 batch 统计量

(3) 层归一化（第 2 层净输入 z ）：



[OK] Z 归一化适用于输入层， θ 归一化 1

[OK] 批量归一化适用于隐藏层， θ 归一化 1

[OK] 层归一化适用于隐藏层， θ 归一化 1

4. 网络优化器对比实验

任务描述：

$$L(\theta_1, \theta_2) = \theta_1^2 - \theta_2^2$$

鞍点位置：(0, 0)

$$\alpha = 0.1 \quad \theta_0 = (0.5, 0.5)$$

优化器公式：

SGD:

$$\theta^{(k+1)} = \theta^{(k)} - \alpha * g^{(k)}$$

RMSprop:

$$v^{(k)} = \beta * v^{(k-1)} + (1-\beta) * (g^{(k)})^2$$

$$\theta^{(k+1)} = \theta^{(k)} - \alpha * g^{(k)} / \sqrt{v^{(k)} + \epsilon}$$

AdaDelta:

$$v^{(k)} = \beta * v^{(k-1)} + (1-\beta) * (g^{(k)})^2$$

$$\Delta \theta^{(k)} = -\sqrt{u^{(k-1)} + \epsilon} / \sqrt{v^{(k)} + \epsilon} * g^{(k)}$$

$$u^{(k)} = \beta * u^{(k-1)} + (1-\beta) * (\Delta \theta^{(k)})^2$$

Adam:

$$m^{(k)} = \beta_1 * m^{(k-1)} + (1-\beta_1) * g^{(k)}$$

$$v^{(k)} = \beta_2 * v^{(k-1)} + (1-\beta_2) * (g^{(k)})^2$$

$$\theta^{(k+1)} = \theta^{(k)} - \alpha * m^{(k)} / \sqrt{v^{(k)} + \epsilon}$$

逃离鞍点速度对比：

优化器	逃离步数	分析
SGD	3	梯度下降直接逃离，但震荡较大
RMSprop	1	自适应学习率使其快速逃离鞍点
AdaDelta	>100	无学习率参数，在鞍点附近移动缓慢
Adam	3	结合动量和自适应学习率，逃离迅速

结果分析：

1. RMS|

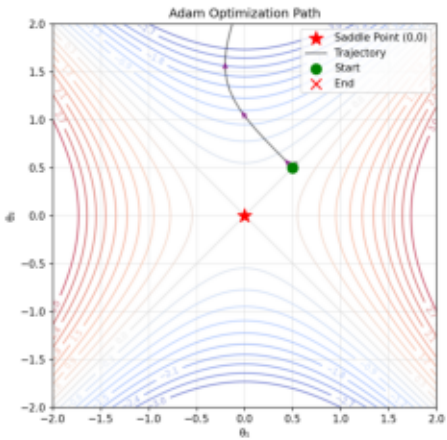
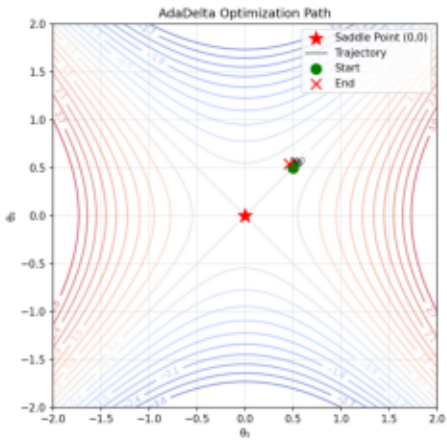
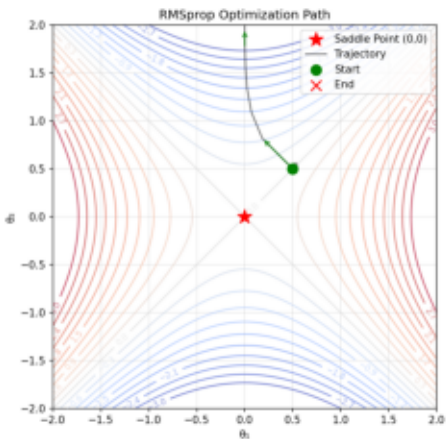
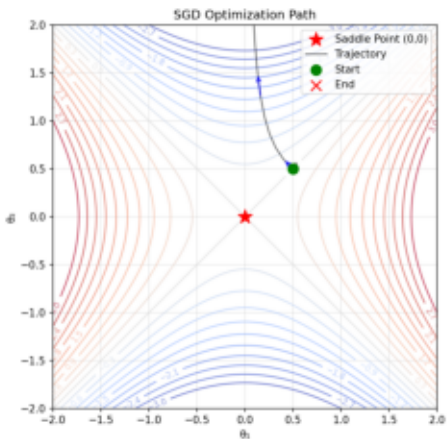
□□□□

2. SGD

□□□□

3. Adal

□□□□



AI 的使用方式

本次作业中 AI 的使用方式如下：

- 1. 在代码文件夹下运行problem1_seq2seq.py, problem2_gcn.py, problem3_normalization.py, problem4_optimizer.py 调用 AI 接口
- 2. 在代码文件夹下运行Problem 2 调用 AI 接口
- 3. 在代码文件夹下运行 AI 接口
- 4. 在代码文件夹下运行 PDF 调用 AI 接口 matplotlib 接口

在代码文件夹下

- 在代码文件夹下
- 在代码文件夹下 AI 接口
- 在代码文件夹下

提交说明：

- 1. 在 PDF 文件中运行 problem*.py 文件
- 2. 在代码文件夹下 weilong@fudan.edu.cn 25112030004@m.fudan.edu.cn
- 3. 在代码文件夹下 - 在 - 在 2 文件夹